Final Exam

As the 2024 Formula 1 season comes to an end, now is the perfect time to take a closer look at the data gathered during the 24 races of the season. This analysis focuses primarily on drivers—their successes and failures—but also examines constructors and other elements that are crucial to the world of Formula 1.

Data Sources

The data used in this project comes from three different sources:

1. **Ergast API**: Accessed through the `ergast-py` library, which offers comprehensive data about the Formula 1 2024 season.
2. **Wikipedia API**: Accessed via the `wikipediaapi` library, providing structured information from Wikipedia.
3. **StatsF1 Website**: A rich repository of Formula 1 data available at StatsF1. This data was collected using the BeautifulSoup library for web scraping.

Ethical Considerations Regarding Data Sources

1. Ergast API

As mentioned on the Ergast website under the Terms & Conditions section, the API may be used for "personal, non-commercial applications and services including educational and research purposes." Moreover, it is prohibited to charge money for applications or services that utilize this API, and each user is obligated to use it responsibly in accordance with the rules and regulations provided on the website.

The purpose of this project is strictly educational, and no money will be charged to recipients of this project. Furthermore, every use of the Ergast-py API was carefully analyzed in terms of compliance with the regulations to ensure that all rules are respected—particularly the rules limiting API calls to no more than 4 requests per second and 200 requests per day.

Taking into account all the aspects listed above, I (as the author) do not have any ethical objections to the use of the API in this project and firmly believe that it aligns with both the letter and the spirit of the regulations governing its use.

2. Wikipedia API

As stated on the official Wikimedia Foundation website in the "API Terms" section, they provide an official set of APIs that are publicly available to "promote free knowledge." Additionally, for individuals interested in using Wikipedia APIs, a comprehensive set of guidelines is available. These guidelines include, for example, respecting request limits and ensuring that requests are sent sequentially rather than in parallel.

This project clearly falls under the category of promoting knowledge. While using the Wikipedia API, I have taken great care to adhere to these guidelines and utilize the API as recommended by its creators. Furthermore, I believe that my use of the API aligns with the ethical principles outlined by the Wikimedia Foundation and my personal ethics. I find this use to be free from any ethical or moral controversy.

3. StatsF1.com Website

Before gathering data, the `robots.txt` file was carefully reviewed to ensure compliance with the website's guidelines. The file explicitly permitted access to all catalogues and subcatalogues for all users except for a few specified bots. Since no restrictions were mentioned that would prohibit data extraction using BeautifulSoup, I concluded that accessing the data in this manner aligns with the website's policies. Therefore, I find no ethical objections to using BeautifulSoup for this purpose.

Dataset

The dataset created for this research (`df_f1_final_data`) contains 23 rows. Each row corresponds to one driver who participated in the 2024 Formula 1 season. The DataFrame includes the following variables:

Variables from Ergast_py API

- **Position** (type: `int`):
  The overall position of the driver in the 2024 season.
- **Driver_Name** (type: `string`):
  The driver's full name.
- **Points** (type: `float`):
  The number of points collected by the driver throughout the 2024 season.
- **Wins** (type: `int`):
  The number of races won by the driver during the season.
- **Constructor** (type: `string`):
  The name of the team the driver competed for in 2024.
- **Positions** (type: `list of int`):
  A list of positions achieved by the driver in all 24 races, ordered chronologically (note: some drivers did not participate in every race, so their lists are shorter than 24).
- **Grids** (type: `list of int`):
  A list of starting grid positions for each race, ordered chronologically (note: some drivers did not participate in every race, so their lists are shorter than 24).
- **Statuses** (type: `list of strings`):
  A list indicating the race status (finished/not finished) for each race in 2024 (note: some drivers did not participate in every race, so their lists are shorter than 24).
- **Top_10_Finishes_Count** (type: `int`):
  The number of races in which the driver finished in the top 10. This is important because drivers earn points only for the top 10 finishes.
- **Total_Races_Count** (type: `int`):
  The number of races each driver participated in during the 2024 season, based on the `Positions` variable.
- **Top_10_Finish_Percentage** (type: `float`):
  The percentage of races in which the driver finished in the top 10 (`Top_10_Finishes_Count` /

`Total_Races_Count` * 100%). This provides a fair metric for comparing drivers' performances, accounting for differences in participation.

- **Finished_Races_Count** (type: `int`):
  The number of races completed by the driver in the season, based on the `Statuses` variable.

- **Finished_Races_Percentage** (type: `float`):
  The percentage of races completed by each driver, based on the `Finished_Races_Count` variable.

- **Most_Frequent_Starting_Position** (type: `int`):
  The most common starting grid position for the driver, based on the `Grids` variable.

- **Pole_Positions_Count** (type: `int`):
  The number of pole positions achieved (i.e., starting a race in 1st place), based on the `Grids` variable.

- **Won_At_Least_One_Race** (type: `int`):
  A binary variable indicating whether the driver won at least one race (`1 = yes, 0 = no`), based on the `Positions` variable.

Variables from Wikipedia API

- **Driver_Name** (type: `string`):
  The driver's full name.

- **Wiki_Summary** (type: `string`):
  The first 100 characters of the driver's Wikipedia summary.

- **Wiki_URL** (type: `string`):
  The URL of the driver's Wikipedia page.

- **Wiki_Categories** (type: `list of strings`):
  Categories listed on the driver's Wikipedia page.

- **Wiki_Languages_Count** (type: `int`):
  The number of languages the driver's Wikipedia page is available in.

- **Processed_Wiki_Summary** (type: `string`):
  A processed version of the Wikipedia summary, converted to lowercase and stripped of non-words.

- **Word_Frequency** (type: `dictionary`):
  A count of each word in the `Processed_Wiki_Summary`.

- **Most_Common_Wiki_Word** (type: `string`):
  The most frequently occurring word in the `Processed_Wiki_Summary`.

- **Most_Common_Wiki_Word_Count** (type: `int`):
  The number of times the most common word appears in the `Processed_Wiki_Summary`.

Variables from StatsF1 Website

- **Driver_Name** (type: `string`):
  The driver's full name.

- **Championship_Titles** (type: `int`):
  The number of championship titles the driver has won.

- **Is_Champion** (type: `int`):
  A binary variable indicating whether the driver has won a championship title (`1 = yes, 0 = no`).

- **First_GP_Year** (type: `string`):
  The year the driver first participated in Formula 1.

- **Years_Of_Experience** (type: `int`):
  The number of years the driver has participated in Formula 1, based on the `First_GP_Year` variable.

- **Nationality** (type: `string`):
  The driver's nationality.

- **Driver_Number** (type: `int`):
  The driver's official racing number.

ver's nationality.

- **Driver Number** (type: `int`):
  The driver's official racing number.

Gathering Data

Installing and Importing Libraries

In [1]:
```
!pip install ergast-py
!pip install pandas
!pip install Counter
!pip install requests
!pip install beautifulsoup4
!pip install numpy
!pip3 install wikipedia-api
!pip install chardet
!pip install unidecode
!pip install matplotlib
!pip install seaborn
!pip install statsmodels

import ergast_py
import pandas as pd
from collections import Counter
import requests
import copy
from bs4 import BeautifulSoup
import re
import numpy as np
import wikipediaapi
import chardet
from unidecode import unidecode
import time
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.formula.api import ols
import statsmodels.formula.api as smf
```

```
Requirement already satisfied: ergast-py in c:\users\user\anaconda3\envs\sds\lib\site-packages
(1.0.0)
Requirement already satisfied: pytz<2023.0,>=2022.5 in c:\users\user\anaconda3\envs\sds\lib\si
te-packages (from ergast-py) (2022.7.1)
Requirement already satisfied: requests<3.0.0,>=2.28.2 in c:\users\user\anaconda3\envs\sds\lib
\site-packages (from ergast-py) (2.32.3)
Requirement already satisfied: uritemplate<5.0.0,>=4.1.1 in c:\users\user\anaconda3\envs\sds\l
ib\site-packages (from ergast-py) (4.1.1)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\user\anaconda3\envs\sds\li
b\site-packages (from requests<3.0.0,>=2.28.2->ergast-py) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\user\anaconda3\envs\sds\lib\site-packa
ges (from requests<3.0.0,>=2.28.2->ergast-py) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\user\anaconda3\envs\sds\lib\site
-packages (from requests<3.0.0,>=2.28.2->ergast-py) (2.2.2)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\user\anaconda3\envs\sds\lib\site
-packages (from requests<3.0.0,>=2.28.2->ergast-py) (2024.7.4)
Requirement already satisfied: pandas in c:\users\user\anaconda3\envs\sds\lib\site-packages
(2.2.2)
Requirement already satisfied: numpy>=1.26.0 in c:\users\user\anaconda3\envs\sds\lib\site-pack
ages (from pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\user\anaconda3\envs\sds\lib
\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\user\anaconda3\envs\sds\lib\site-packa
ges (from pandas) (2022.7.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\user\anaconda3\envs\sds\lib\site-pac
kages (from pandas) (2023.3)
Requirement already satisfied: six>=1.5 in c:\users\user\anaconda3\envs\sds\lib\site-packages
(from python-dateutil>=2.8.2->pandas) (1.16.0)
Collecting Counter
  Downloading Counter-1.0.0.tar.gz (5.2 kB)
  Preparing metadata (setup.py): started
  Preparing metadata (setup.py): finished with status 'done'
Building wheels for collected packages: Counter
  Building wheel for Counter (setup.py): started
  Building wheel for Counter (setup.py): finished with status 'done'
  Created wheel for Counter: filename=Counter-1.0.0-py3-none-any.whl size=5424 sha256=13b648fb
1c0b62ead8b28bd4d0a7aa1dde50a35ef6ccf2d444b76ff2f2a7f20b
  Stored in directory: c:\users\user\appdata\local\pip\cache\wheels\16\ff\7a\6e8bf2fdadb47c50a
03bb4b9a59bd2b1da1b876faf8e3815d9
Successfully built Counter
Installing collected packages: Counter
Successfully installed Counter-1.0.0
Requirement already satisfied: requests in c:\users\user\anaconda3\envs\sds\lib\site-packages
(2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\user\anaconda3\envs\sds\li
b\site-packages (from requests) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\user\anaconda3\envs\sds\lib\site-packa
ges (from requests) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\user\anaconda3\envs\sds\lib\site
-packages (from requests) (2.2.2)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\user\anaconda3\envs\sds\lib\site
-packages (from requests) (2024.7.4)
Requirement already satisfied: beautifulsoup4 in c:\users\user\anaconda3\envs\sds\lib\site-pac
kages (4.12.3)
Requirement already satisfied: soupsieve>1.2 in c:\users\user\anaconda3\envs\sds\lib\site-pack
ages (from beautifulsoup4) (2.5)
Requirement already satisfied: numpy in c:\users\user\anaconda3\envs\sds\lib\site-packages (1.
26.4)
Requirement already satisfied: wikipedia-api in c:\users\user\anaconda3\envs\sds\lib\site-pack
ages (0.7.1)
Requirement already satisfied: requests in c:\users\user\anaconda3\envs\sds\lib\site-packages
(from wikipedia-api) (2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\user\anaconda3\envs\sds\li
b\site-packages (from requests->wikipedia-api) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\user\anaconda3\envs\sds\lib\site-packa
ges (from requests->wikipedia-api) (3.7)
```

```
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\user\anaconda3\envs\sds\lib\site
-packages (from requests->wikipedia-api) (2.2.2)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\user\anaconda3\envs\sds\lib\site
-packages (from requests->wikipedia-api) (2024.7.4)
Requirement already satisfied: chardet in c:\users\user\anaconda3\envs\sds\lib\site-packages
(5.2.0)
Requirement already satisfied: unidecode in c:\users\user\anaconda3\envs\sds\lib\site-packages
(1.3.8)
Requirement already satisfied: matplotlib in c:\users\user\anaconda3\envs\sds\lib\site-package
s (3.8.4)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\user\anaconda3\envs\sds\lib\site-p
ackages (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\user\anaconda3\envs\sds\lib\site-packa
ges (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\user\anaconda3\envs\sds\lib\site-
packages (from matplotlib) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\user\anaconda3\envs\sds\lib\site-
packages (from matplotlib) (1.4.4)
Requirement already satisfied: numpy>=1.21 in c:\users\user\anaconda3\envs\sds\lib\site-packag
es (from matplotlib) (1.26.4)
Requirement already satisfied: packaging>=20.0 in c:\users\user\anaconda3\envs\sds\lib\site-pa
ckages (from matplotlib) (24.1)
Requirement already satisfied: pillow>=8 in c:\users\user\anaconda3\envs\sds\lib\site-packages
(from matplotlib) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\user\anaconda3\envs\sds\lib\site-p
ackages (from matplotlib) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\user\anaconda3\envs\sds\lib\si
te-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\user\anaconda3\envs\sds\lib\site-packages
(from python-dateutil>=2.7->matplotlib) (1.16.0)
Requirement already satisfied: seaborn in c:\users\user\anaconda3\envs\sds\lib\site-packages
(0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\user\anaconda3\envs\sds\lib\si
te-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in c:\users\user\anaconda3\envs\sds\lib\site-packag
es (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\user\anaconda3\envs\sds\lib
\site-packages (from seaborn) (3.8.4)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\user\anaconda3\envs\sds\lib\site-p
ackages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\user\anaconda3\envs\sds\lib\site-packa
ges (from matplotlib!=3.6.1,>=3.4->seaborn) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\user\anaconda3\envs\sds\lib\site-
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\user\anaconda3\envs\sds\lib\site-
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\user\anaconda3\envs\sds\lib\site-pa
ckages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)
Requirement already satisfied: pillow>=8 in c:\users\user\anaconda3\envs\sds\lib\site-packages
(from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\user\anaconda3\envs\sds\lib\site-p
ackages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\user\anaconda3\envs\sds\lib\si
te-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\user\anaconda3\envs\sds\lib\site-packa
ges (from pandas>=1.2->seaborn) (2022.7.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\user\anaconda3\envs\sds\lib\site-pac
kages (from pandas>=1.2->seaborn) (2023.3)
Requirement already satisfied: six>=1.5 in c:\users\user\anaconda3\envs\sds\lib\site-packages
(from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.16.0)
Requirement already satisfied: statsmodels in c:\users\user\anaconda3\envs\sds\lib\site-packag
es (0.14.2)
Requirement already satisfied: numpy>=1.22.3 in c:\users\user\anaconda3\envs\sds\lib\site-pack
ages (from statsmodels) (1.26.4)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in c:\users\user\anaconda3\envs\sds\lib\site
-packages (from statsmodels) (1.14.1)
```

```
Requirement already satisfied: pandas!=2.1.0,>=1.4 in c:\users\user\anaconda3\envs\sds\lib\sit
e-packages (from statsmodels) (2.2.2)
Requirement already satisfied: patsy>=0.5.6 in c:\users\user\anaconda3\envs\sds\lib\site-packa
ges (from statsmodels) (0.5.6)
Requirement already satisfied: packaging>=21.3 in c:\users\user\anaconda3\envs\sds\lib\site-pa
ckages (from statsmodels) (24.1)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\user\anaconda3\envs\sds\lib
\site-packages (from pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\user\anaconda3\envs\sds\lib\site-packa
ges (from pandas!=2.1.0,>=1.4->statsmodels) (2022.7.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\user\anaconda3\envs\sds\lib\site-pac
kages (from pandas!=2.1.0,>=1.4->statsmodels) (2023.3)
Requirement already satisfied: six in c:\users\user\anaconda3\envs\sds\lib\site-packages (from
patsy>=0.5.6->statsmodels) (1.16.0)
```

1. Ergast-py API

In [2]:

```python
# Create an instance of the Ergast class to interact with the Ergast API
e = ergast_py.Ergast()

# Retrieve the driver standings for the 2024 Formula 1 season
driver_standings_list = e.season(2024).get_driver_standings()

# Initialize an empty list to store the extracted data
data = []

# Iterate through the list of standings for each race or season
for standings in driver_standings_list:
    # Access the driver standings within the current standings object
    for standing in standings.driver_standings:
        # Extract data from the DriverStanding object
        position = standing.position  # The driver's position in the standings
        points = standing.points  # Total points scored by the driver
        wins = standing.wins  # Number of races won by the driver
        driver_name = f"{standing.driver.given_name} {standing.driver.family_name}"  # Driver
        constructor_name = standing.constructors[0].name  # Name of the constructor (first in

        # Append the extracted data as a list to the 'data' list
        data.append([position, driver_name, points, wins, constructor_name])

# Create a Pandas DataFrame from the collected data
df = pd.DataFrame(data, columns=['Position', 'Driver_Name', 'Points', 'Wins', 'Constructor'])

# Display the DataFrame
print(df)
```

	Position	Driver_Name	Points	Wins	Constructor
0	1	Max Verstappen	437.0	9	Red Bull
1	2	Lando Norris	374.0	4	McLaren
2	3	Charles Leclerc	356.0	3	Ferrari
3	4	Oscar Piastri	292.0	2	McLaren
4	5	Carlos Sainz	290.0	2	Ferrari
5	6	George Russell	245.0	2	Mercedes
6	7	Lewis Hamilton	223.0	2	Mercedes
7	8	Sergio Pérez	152.0	0	Red Bull
8	9	Fernando Alonso	70.0	0	Aston Martin
9	10	Pierre Gasly	42.0	0	Alpine F1 Team
10	11	Nico Hülkenberg	41.0	0	Haas F1 Team
11	12	Yuki Tsunoda	30.0	0	RB F1 Team
12	13	Lance Stroll	24.0	0	Aston Martin
13	14	Esteban Ocon	23.0	0	Alpine F1 Team
14	15	Kevin Magnussen	16.0	0	Haas F1 Team
15	16	Alexander Albon	12.0	0	Williams
16	17	Daniel Ricciardo	12.0	0	RB F1 Team
17	18	Oliver Bearman	7.0	0	Ferrari
18	19	Franco Colapinto	5.0	0	Williams
19	20	Guanyu Zhou	4.0	0	Sauber
20	21	Liam Lawson	4.0	0	RB F1 Team
21	22	Valtteri Bottas	0.0	0	Sauber
22	23	Logan Sargeant	0.0	0	Williams
23	24	Jack Doohan	0.0	0	Alpine F1 Team

In [3]:
```python
# Create a list to store driver data
driver_data_2 = []

# Iterate through all races in the 2024 season (24 rounds)
for i in range(1, 25):
    # Fetch race results for the given round
    race_result = e.season(2024).round(i).get_results()

    # Add a delay to prevent overwhelming the API
    time.sleep(0.5)

    # Iterate through the race results
    for result in race_result:
        # Check if the 'results' attribute exists in the result object
        if hasattr(result, 'results'):
            for race_detail in result.results:
                # Extract driver details
                driver_name = f"{race_detail.driver.given_name} {race_detail.driver.family_na
                position = race_detail.position  # Final race position
                grid = race_detail.grid  # Starting grid position
                status = 'Finished' if race_detail.status == 1 else 'Not Finished'  # Race st

                # Check if the driver already exists in the dataset
                driver_found = False
                for driver in driver_data_2:
                    if driver[0] == driver_name:
                        # Add race details to the existing driver entry
                        driver[1].append(position)
                        driver[2].append(grid)
                        driver[3].append(status)
                        driver_found = True
                        break

                # If the driver is not found in the dataset, create a new entry
                if not driver_found:
                    driver_data_2.append([driver_name, [position], [grid], [status]])

# Create a DataFrame from the collected driver data
df_2 = pd.DataFrame(driver_data_2, columns=['Driver_Name', 'Positions', 'Grids', 'Statuses'])
```

```python
# Add a column counting the number of races finished in the top 10
df_2['Top_10_Finishes_Count'] = df_2['Positions'].apply(lambda x: sum(1 for pos in x if pos <

# Add a column counting the total number of races a driver participated in
df_2['Total_Races_Count'] = df_2['Positions'].apply(len)

# Add a column calculating the percentage of races finished in the top 10
df_2['Top_10_Finish_Percentage'] = df_2.apply(
    lambda row: round((row['Top_10_Finishes_Count'] / row['Total_Races_Count']) * 100, 2)
    if row['Total_Races_Count'] > 0 else 0,
    axis=1
)

# Add a column counting the number of races finished
df_2['Finished_Races_Count'] = df_2['Statuses'].apply(lambda x: sum(1 for status in x if stat

# Add a column calculating the percentage of races finished
df_2['Finished_Races_Percentage'] = df_2.apply(
    lambda row: round((row['Finished_Races_Count'] / row['Total_Races_Count']) * 100, 2)
    if row['Total_Races_Count'] > 0 else 0,
    axis=1
)

# Add a column identifying the most frequent starting grid position
df_2['Most_Frequent_Starting_Position'] = df_2['Grids'].apply(lambda x: Counter(x).most_commo

# Add a column counting the number of pole positions (grid position = 1)
df_2['Pole_Positions_Count'] = df_2['Grids'].apply(lambda x: sum(1 for pos in x if pos == 1))

# Add a column indicating if the driver won at least one race (position = 1)
df_2['Won_at_Least_One_Race'] = df_2['Positions'].apply(lambda x: 1 if any(pos == 1 for pos i

# Display the final DataFrame
print(df_2.head())
```

```
       Driver_Name                                          Positions  \
0    Max Verstappen  [1, 1, 19, 1, 1, 2, 1, 6, 1, 1, 5, 2, 5, 4, 2,...
1      Sergio Pérez  [2, 2, 5, 2, 3, 4, 8, 18, 18, 8, 7, 17, 7, 7, ...
2      Carlos Sainz  [3, 1, 3, 5, 5, 5, 3, 16, 6, 3, 5, 6, 6, 5, 4,...
3  Charles Leclerc  [4, 3, 2, 4, 4, 3, 3, 1, 19, 5, 11, 14, 4, 3, ...
4   George Russell  [5, 6, 17, 7, 6, 8, 7, 5, 3, 4, 1, 19, 8, 20, ...

                                              Grids  \
0  [1, 1, 1, 1, 1, 1, 1, 6, 2, 2, 1, 4, 3, 11, 2,...
1  [5, 3, 6, 2, 2, 4, 11, 16, 16, 11, 8, 0, 16, 2...
2  [4, 2, 4, 7, 3, 4, 3, 12, 6, 4, 7, 4, 7, 10, 5...
3  [2, 2, 4, 8, 6, 2, 3, 1, 11, 5, 6, 11, 6, 1, 6...
4  [3, 7, 7, 9, 8, 7, 6, 5, 1, 4, 3, 1, 17, 6, 4,...

                                            Statuses  Top_10_Finishes_Count  \
0  [Finished, Finished, Not Finished, Finished, F...                     23
1  [Finished, Finished, Finished, Finished, Finis...                     16
2  [Finished, Finished, Finished, Finished, Finis...                     20
3  [Finished, Finished, Finished, Finished, Finis...                     21
4  [Finished, Finished, Not Finished, Finished, F...                     21

   Total_Races_Count  Top_10_Finish_Percentage  Finished_Races_Count  \
0                 24                     95.83                    23
1                 24                     66.67                    16
2                 23                     86.96                    20
3                 24                     87.50                    22
4                 24                     87.50                    21

   Finished_Races_Percentage  Most_Frequent_Starting_Position  \
0                      95.83                               1
1                      66.67                               2
2                      86.96                               4
3                      91.67                               4
4                      87.50                               1

   Pole_Positions_Count  Won_at_Least_One_Race
0                     8                      1
1                     0                      0
2                     1                      1
3                     3                      1
4                     4                      1
```

2. Wikipedia API

In [4]:

```python
# Initialize Wikipedia API for fetching driver information
wiki_wiki = wikipediaapi.Wikipedia('base_camp (btg443@alumni.ku.dk)', 'en')

# Create a DataFrame with driver names from the original DataFrame
df_3 = pd.DataFrame(df['Driver_Name'], columns=['Driver_Name'])

# Initialize new columns in the DataFrame for Wiki data
df_3['Wiki_Summary'] = ''
df_3['Wiki_Url'] = ''
df_3["Wiki_Categories"] = ''
df_3['Wiki_Languages Count'] = 0

# Replace spaces in driver names with underscores to match Wikipedia page format
df_3['Driver_Name'] = df_3['Driver_Name'].str.replace(' ', '_')

# Iterate over each driver in the DataFrame to fetch their Wikipedia page and details
for i in range(len(df_3)):
    # Fetch the Wikipedia page for the driver
    page_py = wiki_wiki.page(df_3['Driver_Name'][i])

    # Check if the Wikipedia page exists
```

```
    if page_py.exists():
        # Extract the first 1000 characters of the Wikipedia summary
        df_3.at[i, 'Wiki_Summary'] = page_py.summary[0:1000]
        # Get the full URL of the Wikipedia page
        df_3.at[i, 'Wiki_Url'] = page_py.fullurl
        # Count the number of languages the page is available in
        df_3.at[i, 'Wiki_Languages_Count'] = len(page_py.langlinks)
        # Get the categories of the Wikipedia page
        categories = page_py.categories
        # Create a list of categories and join them into a string
        categories_list = [cat for cat in categories]
        df_3.at[i, 'Wiki_Categories'] = ', '.join(categories_list)
    else:
        # If the page does not exist, set default values
        df_3.at[i, 'Wiki_Summary'] = 'No summary available'
        df_3.at[i, 'Wiki_Categories'] = 'No categories available'
        df_3.at[i, 'Wiki_Url'] = 'No url available'

# Replace underscores back to spaces in the driver names for readability
df_3['Driver_Name'] = df_3['Driver_Name'].str.replace('_', ' ')

# Process the Wikipedia summary: convert to lowercase and remove non-alphabetic characters
df_3['Processed_Wiki_Summary'] = df_3['Wiki_Summary'].str.lower().apply(lambda x: re.sub(r'[^

# Count the frequency of each word in the processed summary
df_3['Word_Frequency'] = df_3['Processed_Wiki_Summary'].apply(lambda x: pd.Series(x.split()).

# Identify the most common word in the processed summary
df_3['Most_Common_Wiki_Word'] = df_3['Word_Frequency'].apply(lambda x: max(x, key=x.get) if x

# Count how many times the most common word appears in the processed summary
df_3['Most_Common_Wiki_Word_Count'] = df_3['Word_Frequency'].apply(lambda x: max(x.values()) 

# Display the first few rows of the DataFrame
df_3.head()
```

Out[4]:

	Driver_Name	Wiki_Summary	Wiki_Url	Wiki_Categories	Wiki_Lang
0	Max Verstappen	Max Emilian Verstappen (Dutch pronunciation: [...	https://en.wikipedia.org/wiki/Max_Verstappen	Category:1997 births, Category:21st-century Du...	
1	Lando Norris	Lando Norris (born 13 November 1999) is a Brit...	https://en.wikipedia.org/wiki/Lando_Norris	Category:1999 births, Category:24 Hours of Day...	
2	Charles Leclerc	Charles Marc Hervé Perceval Leclerc (French pr...	https://en.wikipedia.org/wiki/Charles_Leclerc	Category:1997 births, Category:ART Grand Prix ...	
3	Oscar Piastri	Oscar Jack Piastri (born 6 April 2001) is an A...	https://en.wikipedia.org/wiki/Oscar_Piastri	Category:2001 births, Category:21st-century Au...	
4	Carlos Sainz	Carlos Sainz may refer to:	https://en.wikipedia.org/wiki/Carlos_Sainz	Category:All article disambiguation pages, Cat...	

3. StatsF1.com Website

In [5]:
```
# Set up headers for requests to simulate a browser visit
headers = {'User-Agent': 'Mozilla/5.0'}

# Fetch the webpage with driver championship statistics
response = requests.get(url='https://www.statsf1.com/en/statistiques/pilote/champion/nombre.a

# Check the response status
if response.status_code == 200:
    data = response.text
    soup = BeautifulSoup(data, 'html.parser')

    # Find the table containing the driver statistics
    table = soup.find('table', {'class': 'sortable'})

    # If the table is found
    if table:
        rows = table.find_all('tr')  # Get all the rows in the table

        # List to store results
        driver_titles = []

        # Iterate through the rows, skipping the header
        for row in rows[1:]:
            cols = row.find_all('td')
            if len(cols) > 1:
                # Extract driver name and split into first and last names
                driver_name = cols[1].get_text(strip=True)
                driver_name_parts = driver_name.split()  # Split into ['VERSTAPEN', 'Max']
                if len(driver_name_parts) == 2:
                    driver_name = f"{driver_name_parts[1]} {driver_name_parts[0].title()}"  #
                # Get the championship titles count
                championship_count = cols[5].get_text(strip=True)

                # Add the data to the list
                driver_titles.append([driver_name, championship_count])

        # Create a DataFrame from the list of results
        df_drivers_championships = pd.DataFrame(driver_titles, columns=['Driver_Name', 'Champ
    else:
        print("Table not found.")
else:
    print('Error')

# Create a DataFrame with 2024 drivers
df_2024_drivers = pd.DataFrame(df['Driver_Name'], columns=['Driver_Name'])

# Merge the 2024 drivers with the championship data
df_merged = pd.merge(df_2024_drivers, df_drivers_championships, on='Driver_Name', how='left')

# Replace NaN values with 0 for championship titles
df_merged['Championship_Titles'] = df_merged['Championship_Titles'].fillna(0).astype(int)

# Create a binary variable indicating if the driver is a champion
df_merged['Is_Champion'] = (df_merged['Championship_Titles'] > 0).astype(int)

# Print the merged DataFrame
#print(df_merged)

# Now add variables for the first season, nationality, and driver number
drivers = df['Driver_Name']
# List to store the results
results = []
```

```python
# Iterate through the drivers
for driver in drivers:
    # Convert the driver name into a format suitable for the URL
    driver_url_part = unidecode(driver).lower().replace(" ", "-")
    url = f"https://www.statsf1.com/en/{driver_url_part}.aspx"
    time.sleep(0.5)  # Add a small delay to prevent overloading the website

    # Fetch the webpage
    response = requests.get(url, headers=headers)

    if response.status_code == 200:
        # Parse the webpage
        soup = BeautifulSoup(response.text, 'html.parser')

        # Find the element containing the first Grand Prix year
        first_gp_element = soup.find("a", {"id": "ctl00_CPH_Main_HL_FirstGP"})
        if first_gp_element:
            first_gp_text = first_gp_element.text.strip()
            #print(f"Found first GP for {driver}: {first_gp_text}")

            # Extract the year from the text
            year = first_gp_text.split()[-1]
            if not year.isdigit():
                year = None  # If the year is not a valid number
        else:
            print(f"No first GP found for {driver}")
            year = None

        # Find the element containing the driver's nationality
        nationality_element = soup.find("a", {"id": "ctl00_CPH_Main_HL_Pays"})
        if nationality_element:
            nationality = nationality_element.text.strip()
            #print(f"Found nationality for {driver}: {nationality}")
        else:
            print(f"No nationality found for {driver}")
            nationality = None

        # Find the element containing the driver's number
        driver_number_element = soup.find("div", {"id": "ctl00_CPH_Main_P_Numero"})
        if driver_number_element:
            driver_number_span = driver_number_element.find("span")
            if driver_number_span:
                driver_number = driver_number_span.text.strip()
                #print(f"Found driver number for {driver}: {driver_number}")
            else:
                driver_number = None
        else:
            print(f"No driver number found for {driver}")
            driver_number = None

    else:
        print(f"Error fetching data for {driver}: Status code {response.status_code}")
        year = None
        nationality = None
        driver_number = None

    # Add the data to the list of results
    results.append({
        "Driver_Name": driver,
        "First_GP_Year": year,
        "Nationality": nationality,
        "Driver_Number": driver_number
    })

# Create a DataFrame with the scraped data
```

```
df_scraped_data = pd.DataFrame(results)

# Print the DataFrame
#print(df_scraped_data)

# Merge all the scraped data into one DataFrame
df_4 = pd.merge(df_merged, df_scraped_data, on='Driver_Name', how='left')

# Add a column for the years of experience
df_4["Years_of_Experience"] = 2024 - df_4["First_GP_Year"].astype(int)

# Print the final DataFrame
df_4
```

Out[5]:

	Driver_Name	Championship_Titles	Is_Champion	First_GP_Year	Nationality	Driver_Number	Years_
0	Max Verstappen	4	1	2015	Netherlands	33	
1	Lando Norris	0	0	2019	United Kingdom	4	
2	Charles Leclerc	0	0	2018	Monaco	16	
3	Oscar Piastri	0	0	2023	Australia	81	
4	Carlos Sainz	0	0	2015	Spain	55	
5	George Russell	0	0	2019	United Kingdom	63	
6	Lewis Hamilton	7	1	2007	United Kingdom	44	
7	Sergio Pérez	0	0	2011	Mexico	11	
8	Fernando Alonso	2	1	2001	Spain	14	
9	Pierre Gasly	0	0	2017	France	10	
10	Nico Hülkenberg	0	0	2010	Germany	27	
11	Yuki Tsunoda	0	0	2021	Japan	22	
12	Lance Stroll	0	0	2017	Canada	18	
13	Esteban Ocon	0	0	2016	France	31	
14	Kevin Magnussen	0	0	2014	Denmark	20	
15	Alexander Albon	0	0	2019	Thailand	23	
16	Daniel Ricciardo	0	0	2011	Australia	3	
17	Oliver Bearman	0	0	2024	United Kingdom	87	
18	Franco Colapinto	0	0	2024	Argentina	43	
19	Guanyu Zhou	0	0	2022	China	24	
20	Liam Lawson	0	0	2023	New Zealand	30	
21	Valtteri Bottas	0	0	2013	Finland	77	
22	Logan Sargeant	0	0	2023	USA	2	
23	Jack Doohan	0	0	2024	Australia	7	

Merging Data

In [6]:

```
# Merging all the data from the APIs into one table

# Merge df with df_2 based on 'Driver Name'
df_merged_1_2 = pd.merge(df, df_2, on='Driver_Name', how='left')

# Merge the previous result with df_3
df_api = pd.merge(df_merged_1_2, df_3, on='Driver_Name', how='left')

# Display the merged DataFrame
#print(df_api)

# Merging all the data into one final table

# Merge df_api with df_4 based on 'Driver Name'
df_f1_final_data = pd.merge(df_api, df_4, on='Driver_Name', how='left')

# Display the final merged DataFrame
print(df_f1_final_data.head())

# Saving the final DataFrame to a CSV file
df_f1_final_data.to_csv('f1_final_data.csv', sep=';', index=False)
```

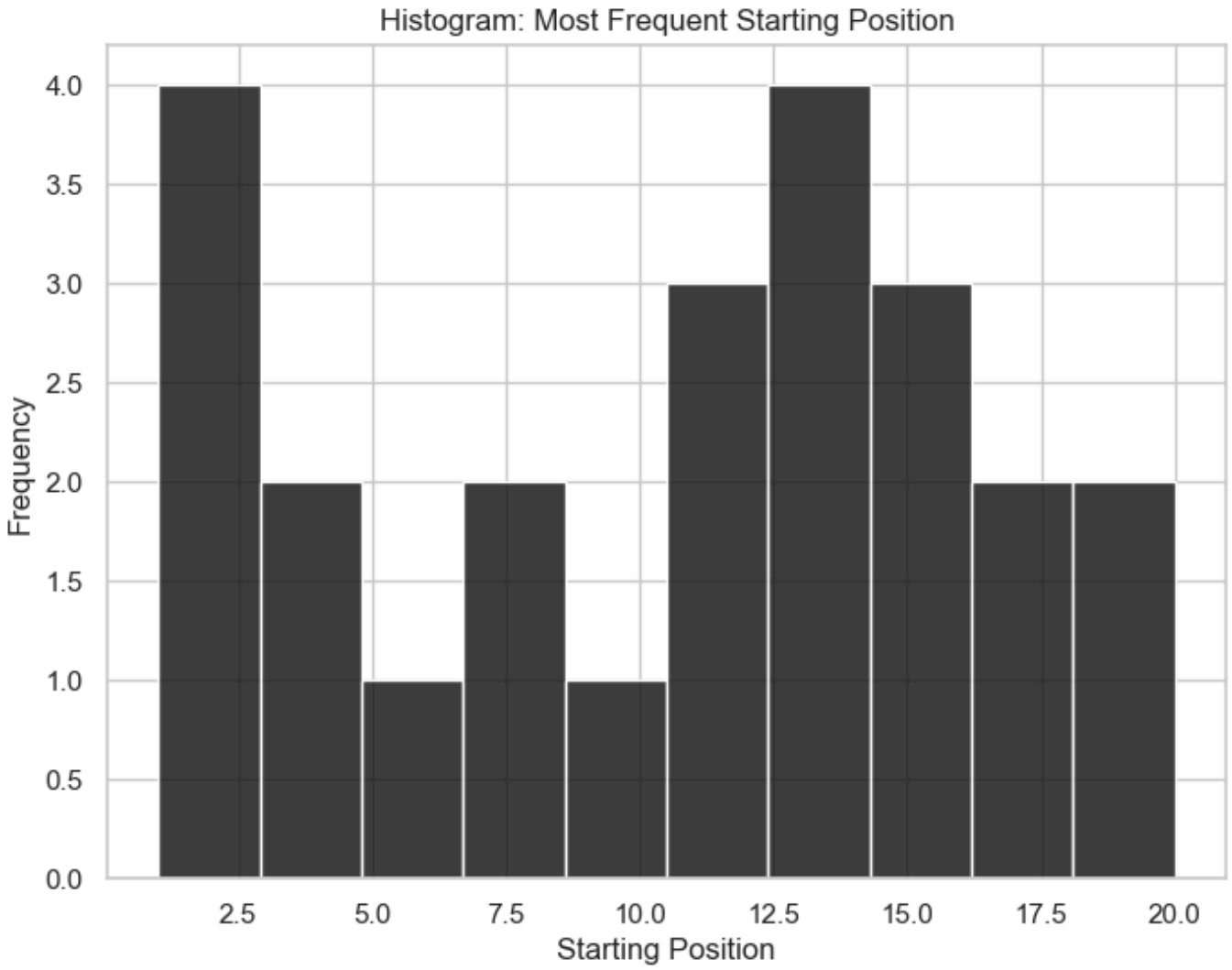
```
   Position      Driver_Name  Points  Wins Constructor  \
0         1   Max Verstappen   437.0     9    Red Bull
1         2     Lando Norris   374.0     4     McLaren
2         3  Charles Leclerc   356.0     3     Ferrari
3         4    Oscar Piastri   292.0     2     McLaren
4         5     Carlos Sainz   290.0     2     Ferrari

                                          Positions  \
0  [1, 1, 19, 1, 1, 2, 1, 6, 1, 1, 5, 2, 5, 4, 2,...
1  [6, 8, 3, 5, 2, 1, 2, 4, 2, 2, 20, 3, 2, 5, 1,...
2  [4, 3, 2, 4, 4, 3, 3, 1, 19, 5, 11, 14, 4, 3, ...
3  [8, 4, 4, 8, 8, 13, 4, 2, 5, 7, 2, 4, 1, 2, 4,...
4  [3, 1, 3, 5, 5, 5, 3, 16, 6, 3, 5, 6, 6, 5, 4,...

                                             Grids  \
0  [1, 1, 1, 1, 1, 1, 1, 6, 2, 2, 1, 4, 3, 11, 2,...
1  [7, 6, 3, 3, 4, 5, 2, 4, 3, 1, 2, 3, 1, 4, 1, ...
2  [2, 2, 4, 8, 6, 2, 3, 1, 11, 5, 6, 11, 6, 1, 6...
3  [8, 5, 5, 6, 5, 6, 5, 2, 4, 9, 7, 5, 2, 5, 3, ...
4  [4, 2, 4, 7, 3, 4, 3, 12, 6, 4, 7, 4, 7, 10, 5...

                                           Statuses  Top_10_Finishes_Count  \
0  [Finished, Finished, Not Finished, Finished, F...                     23
1  [Finished, Finished, Finished, Finished, Finis...                     23
2  [Finished, Finished, Finished, Finished, Finis...                     21
3  [Finished, Finished, Finished, Finished, Finis...                     23
4  [Finished, Finished, Finished, Finished, Finis...                     20

   Total_Races_Count  ...                            Processed_Wiki_Summary  \
0                 24  ...  max emilian verstappen dutch pronunciation mks...
1                 24  ...  lando norris born  november  is a british raci...
2                 24  ...  charles marc herv perceval leclerc french pron...
3                 24  ...  oscar jack piastri born  april  is an australi...
4                 23  ...                          carlos sainz may refer to

                                      Word_Frequency  Most_Common_Wiki_Word  \
0  {'in': 8, 'the': 6, 'verstappen': 6, 'formula'...                     in
1  {'in': 12, 'the': 8, 'formula': 7, 'norris': 6...                     in
2  {'in': 11, 'the': 8, 'leclerc': 6, 'formula': ...                     in
3  {'in': 9, 'formula': 9, 'the': 7, 'piastri': 6...                     in
4  {'carlos': 1, 'sainz': 1, 'may': 1, 'refer': 1...                 carlos

   Most_Common_Wiki_Word_Count  Championship_Titles  Is_Champion  \
0                            8                    4            1
1                           12                    0            0
2                           11                    0            0
3                            9                    0            0
4                            1                    0            0

  First_GP_Year     Nationality Driver_Number  Years_of_Experience
0          2015     Netherlands            33                    9
1          2019  United Kingdom             4                    5
2          2018          Monaco            16                    6
3          2023       Australia            81                    1
4          2015           Spain            55                    9

[5 rows x 31 columns]
```

Data Analysis

- Visualizations

- **Interval Outome Variables**
  - Most Frequent Starting Position

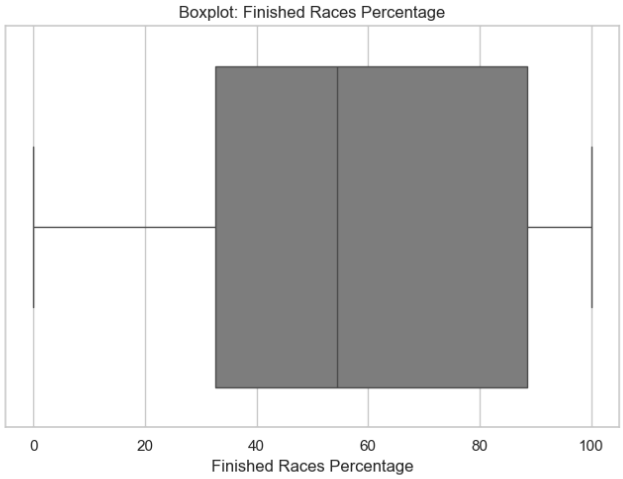
In [7]:
```
# Set the theme for Matplotlib visualizations
plt.style.use('grayscale')

# Set the theme for Seaborn visualizations
sns.set_theme(style="whitegrid", palette="gray")

# **1. Interval Outcome Variable**
# Visualizing 'Most Frequent Starting Position' with a histogram
plt.figure(figsize=(8, 6))
sns.histplot(df_f1_final_data['Most_Frequent_Starting_Position'], bins=10, kde=False, color="
plt.title('Histogram: Most Frequent Starting Position')  # Set title for the plot
plt.xlabel('Starting Position')  # Label for the x-axis
plt.ylabel('Frequency')  # Label for the y-axis
plt.show()
```

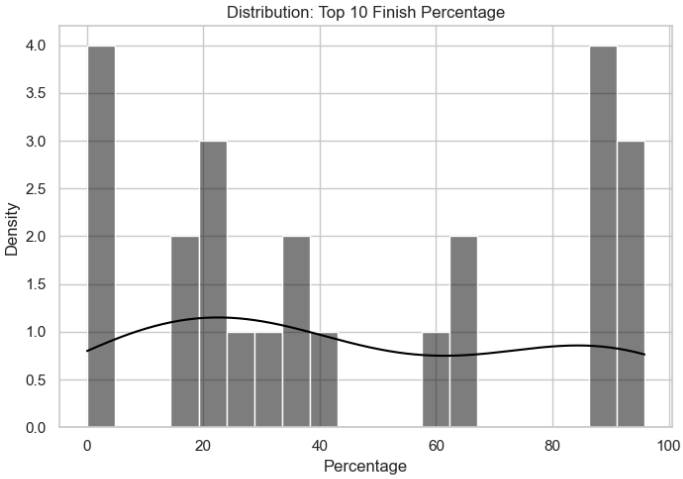


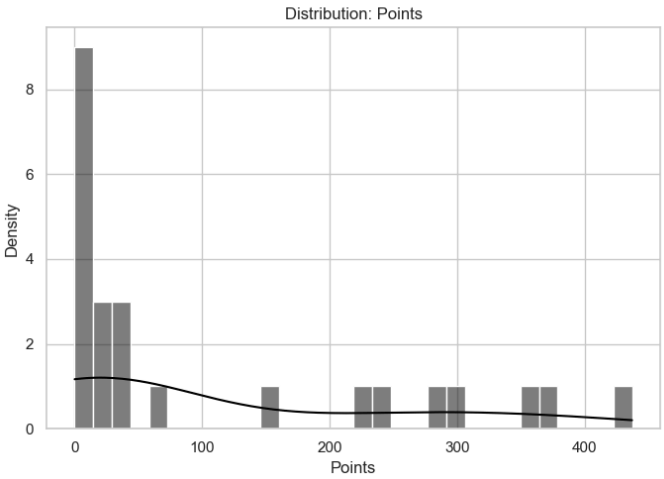
- **Continuous Outcome Variables**
  - Finished Races Percentage
  - Top 10 Finish Percentage
  - Points
  - Wins

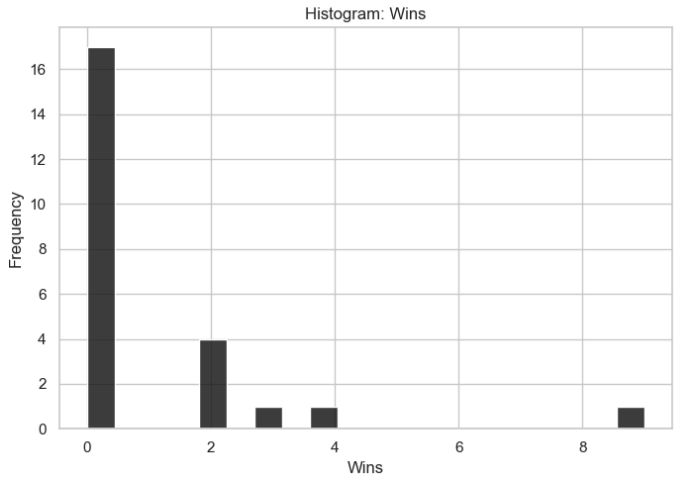
In [8]:
```
# Setting up the figure for 2x2 subplots
fig, axes = plt.subplots(2, 2, figsize=(14, 10))  # 2x2 layout, larger figure size for clarity

# Plot 1: Boxplot for 'Finished Races Percentage'
sns.boxplot(
    x=df_f1_final_data['Finished_Races_Percentage'],
    color="gray",
    ax=axes[0, 0]
```

```
)
axes[0, 0].set_title('Boxplot: Finished Races Percentage')  # Title for the plot
axes[0, 0].set_xlabel('Finished Races Percentage')  # Label for the x-axis

# Plot 2: Distribution plot for 'Top 10 Finish Percentage'
sns.histplot(
    df_f1_final_data['Top_10_Finish_Percentage'],
    bins=20,
    kde=True,
    color="black",
    ax=axes[0, 1]
)
axes[0, 1].set_title('Distribution: Top 10 Finish Percentage')  # Title for the plot
axes[0, 1].set_xlabel('Percentage')  # Label for the x-axis
axes[0, 1].set_ylabel('Density')  # Label for the y-axis

# Plot 3: Histogram with KDE for 'Points'
sns.histplot(
    df_f1_final_data['Points'],
    bins=30,
    kde=True,
    color="black",
    ax=axes[1, 0]
)
axes[1, 0].set_title('Distribution: Points')  # Set title for the plot
axes[1, 0].set_xlabel('Points')  # Label for the x-axis
axes[1, 0].set_ylabel('Density')  # Label for the y-axis

# Plot 4: Histogram for 'Wins'
sns.histplot(
    df_f1_final_data['Wins'],
    bins=20,
    kde=False,
    color="black",
    ax=axes[1, 1]
)
axes[1, 1].set_title('Histogram: Wins')  # Set title for the plot
axes[1, 1].set_xlabel('Wins')  # Label for the x-axis
axes[1, 1].set_ylabel('Frequency')  # Label for the y-axis

# Adjust layout to prevent overlapping
plt.tight_layout()

# Display the plot
plt.show()
```





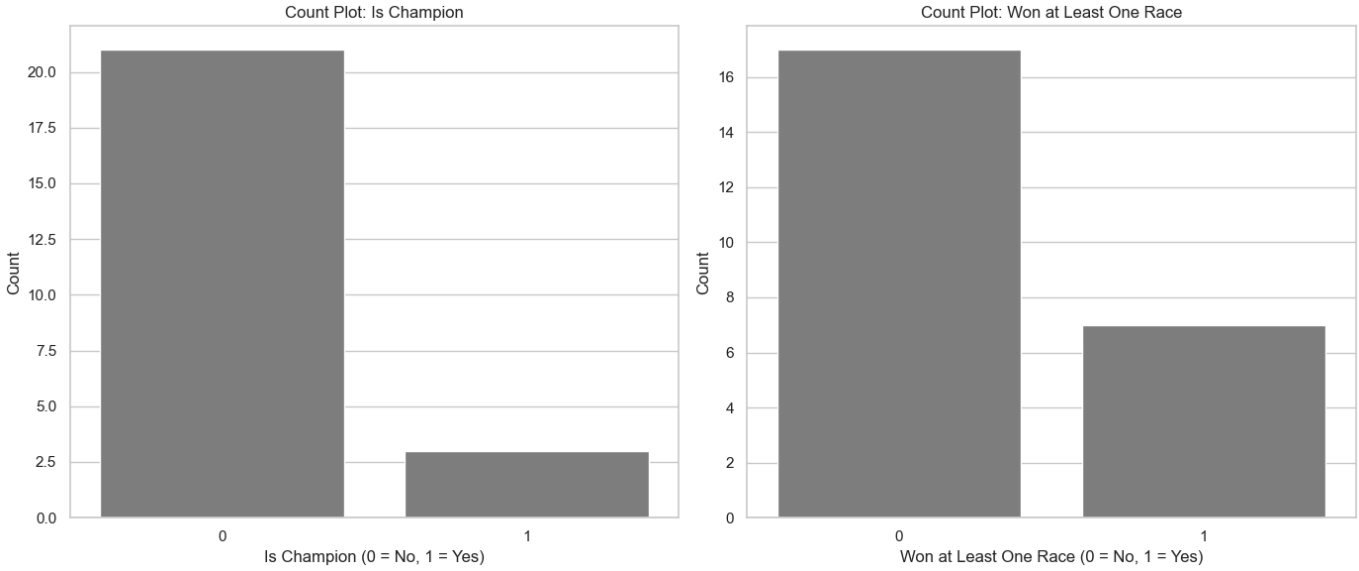




- **Binary Outcome Variables**
  - Is Champion
  - Won at Least One Race

In [10]:

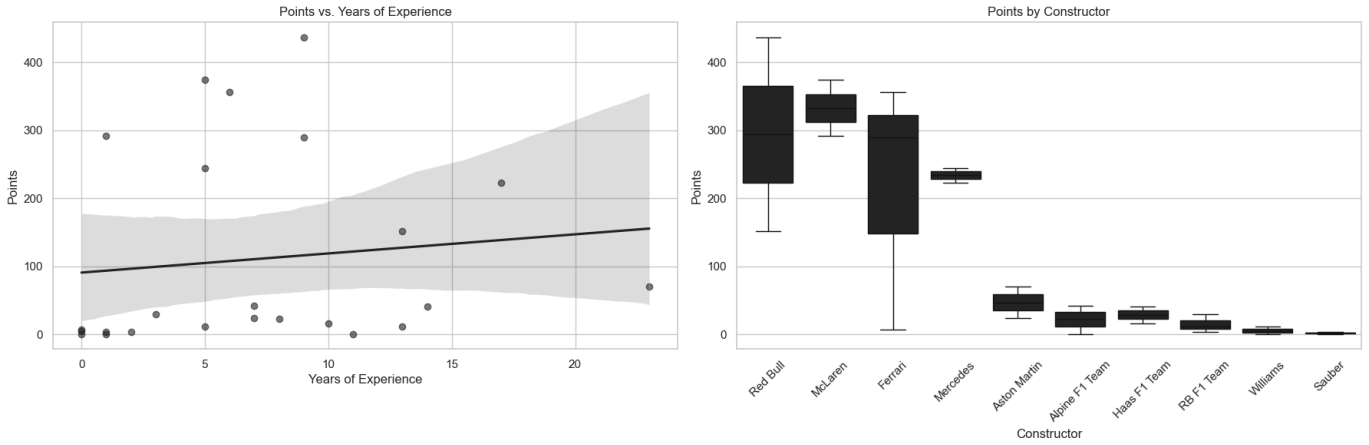
```
# Setting up the figure for 1x2 subplots
fig, axes = plt.subplots(1, 2, figsize=(14, 6))  # 1x2 layout, larger figure size for clarity

# Plot 1: Bar plot for 'Is Champion'
sns.countplot(
    x=df_f1_final_data['Is_Champion'],
    color="gray",
    ax=axes[0]
)
axes[0].set_title('Count Plot: Is Champion')  # Set title for the plot
axes[0].set_xlabel('Is Champion (0 = No, 1 = Yes)')  # Label for the x-axis
axes[0].set_ylabel('Count')  # Label for the y-axis

# Plot 2: Bar plot for 'Won at Least One Race'
sns.countplot(
    x=df_f1_final_data['Won_at_Least_One_Race'],
    color="gray",
    ax=axes[1]
)
axes[1].set_title('Count Plot: Won at Least One Race')  # Set title for the plot
axes[1].set_xlabel('Won at Least One Race (0 = No, 1 = Yes)')  # Label for the x-axis
axes[1].set_ylabel('Count')  # Label for the y-axis

# Adjust layout to prevent overlapping
plt.tight_layout()

# Display the plot
plt.show()
```



- **Making two bivariate plots containing the outcome variable and two predictors (one predictor per plot):**
  - **Plot 1**
    - outome variable: Points
    - predictor: Years_Of_Experience
  - **Plot 2**
    - outcome variable: Points
    - predictor: Constructor

In [12]:
```
# Creating a figure with two subplots placed side by side using plt.subplots()
# figsize=(18, 6) sets the overall size of the figure: width of 18 inches and height of 6 incl
fig, axes = plt.subplots(1, 2, figsize=(18, 6))


# Plot 1: Scatter Plot for Points vs. Years_Of_Experience

# Creating a scatter plot to visualize the relationship between the outcome variable (Points)
# and a numerical predictor (Years_Of_Experience), with a regression line to check the linear
sns.regplot(x='Years_of_Experience', y='Points', data=df_f1_final_data,
            scatter_kws={'alpha': 0.6}, ax=axes[0])

# Adding a title to the first subplot to clarify what it represents
axes[0].set_title('Points vs. Years of Experience')

# Labeling the x-axis to indicate the numerical predictor (Years_Of_Experience)
axes[0].set_xlabel('Years of Experience')

# Labeling the y-axis to indicate the outcome variable (Points)
axes[0].set_ylabel('Points')


# Plot 2: Boxplot for Points by Constructor

# Creating a boxplot to visualize the distribution of the outcome variable (Points)
# across different categories of the predictor variable (Constructor)
sns.boxplot(x='Constructor', y='Points', data=df_f1_final_data, ax=axes[1])

# Adding a title to the second subplot to clarify what it represents
axes[1].set_title('Points by Constructor')

# Labeling the x-axis to indicate the categorical predictor (Constructor)
axes[1].set_xlabel('Constructor')
```

```
# Labeling the y-axis to indicate the outcome variable (Points)
axes[1].set_ylabel('Points')

# Rotating the x-axis labels for better readability, especially when there are many categories
axes[1].tick_params(axis='x', rotation=45)

# Using tight_layout() to ensure that the subplots don't overlap and have proper spacing betwe
plt.tight_layout()

# Displaying the final figure with both subplots
plt.show()
```



- Regressions

Model 1

- A regression model for one of your continuous/interval outcome variables

and a predictor of interest (e.g. predicting the number of likes a tweet receives as a function of the number of a user's followes) as the predictor.

- **Outcome Variable**: Points
- **Predictor**: Years_Of_Experience

In [13]:
```
est_mod_1 = ols('Points ~ Years_of_Experience',data=df_f1_final_data).fit()
print(np.sqrt(est_mod_1.scale))
est_mod_1.summary().tables[1]
```

```
146.11940538422598
```

Out[13]:

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	90.8643	46.865	1.939	0.065	-6.327	188.055
Years_of_Experience	2.8133	5.103	0.551	0.587	-7.770	13.397

- **Interpretation of Results:**

$$Y_i = 90.86 + 2.81X_i + \epsilon_i$$

- **beta 0 = 90.86**: A Formula 1 driver in 2024 season was expected to score 90.86 points during the whole season when having 0 years of experience in Formula 1 (Years_of_Experience = 0). The p-value for the intercept is 0.065, which is greater than the typical significance threshold of 0.05. This means that we do not have sufficient evidence to consider the intercept statistically significant. It means that the baseline value (90.86 points) for a driver with 0 years of experience may not be reliable and could be the

result of random chance rather than a meaningful relationship.
- **beta 1 = 2.81**: For each one-unit increase in Years_Of_ Experience (additional year of Formula 1 experience), it is expected for a driver to score on average an additional 2.81 more points. The p-value for the coefficient of "Years_of_Experience" is 0.587, which is much greater than the typical significance threshold of 0.05. This indicates that we do not have sufficient evidence to consider the relationship between years of experience and points as statistically significant. There is a high probability that the observed effect (an increase of 2.81 points) is due to random chance, rather than a true relationship between experience and performance.
- **The RMSE value = 146.12**: This indicates that, on average, the model's predictions deviate by 146.12 points from the actual values. This suggests a relatively high level of prediction error, indicating that additional predictors or a different modeling approach might improve the model's ability to explain the variation in points earned.

- **Squared Term And Log Transformation**

Squared Term And Log Transformation are both ways of modifying the variables to address non-linearity. However, log transformations are particularly useful when the relationship between the predictor and outcome is multiplicative or exponential, such as in cases of growth or when dealing with highly skewed data. On the other hand, squared terms are typically used to capture quadratic relationships between the predictor and the outcome. This is useful when the relationship appears to follow a curved, but symmetric U-shaped or inverted U-shaped pattern, indicating that the effect of the predictor on the outcome might be increasing up to a point and then decreasing (or vice versa). To investigate whether a squared term for any of the predictors should be included or whether the log ofther outcome or any ofther predictor should be taken it is crucial to examine the distributions of the variables.

In [22]:
```
# Performing a Residual Plot to check if residuals are randomly scattered around the horizonta
# If the residuals are randomly scattered around the horizontal line at 0, it suggests that th
# between the predictor and outcome variable is linear.
residuals = est_mod_1.resid  # Extracting the residuals from the fitted regression model

# Creating a scatter plot of residuals vs 'Years_of_Experience'
# This helps us visually inspect if the residuals follow a random pattern or exhibit any spec
plt.scatter(df_f1_final_data['Years_of_Experience'], residuals)

# Adding a horizontal line at 0 to represent the baseline where residuals should be centered
# The line helps us see if residuals are evenly distributed around 0 (indicating a linear rela
plt.axhline(0, color='red', linestyle='--')

# Labeling the x-axis as 'Years of Experience' and the y-axis as 'Residuals' for clarity
plt.xlabel('Years of Experience')
plt.ylabel('Residuals')

# Adding a title to the plot to indicate what it represents
plt.title('Residual Plot: Years of Experience vs Residuals')

# Displaying the plot
plt.show()

# Analysis of the Residual Plot:
# Upon examining the plot, we can observe that for drivers with 15 to 20 years of experience,
# and the residuals are not randomly scattered around the horizontal line at 0.
# This suggests that the relationship between 'Years of Experience' and the 'Points' outcome v
# Therefore, it would be worth considering non-linear transformations.
```

The residual plot reveals a non-linear relationship between the predictor variable ('Years of Experience') and the outcome variable ('Points'). To further investigate whether the data points exhibit any systematic pattern or are skewed, a scatter plot will be generated. This plot will help visualize the relationship between 'Years of Experience' and 'Points' more clearly and provide insights into the distribution and potential irregularities in the data.

In [6]:
```
# Creating a scatter plot to investigate the relationship between 'Years_of_Experience' and 'P
# Scatter plots provide a clear visual representation of how the predictor and outcome variab

plt.scatter(df_f1_final_data['Years_of_Experience'], df_f1_final_data['Points'], color='black

# Labeling the x-axis to indicate the predictor variable
plt.xlabel('Years of Experience')

# Labeling the y-axis to indicate the outcome variable
plt.ylabel('Points')

# Adding a title to provide context for the plot
plt.title('Scatter Plot: Years of Experience vs Points')

# Displaying the plot to analyze patterns, skewness, or any irregularities in the data
plt.show()
```

In this case the data do not exhibit a linear relationship and also do not follow any discernible regular pattern, such as symmetry or exponential trends. I would not apply squared term or log transformation to this model. However, I would consider applying other non-linear tranformations and also including other predictors in this model to see if there are any interaction effects.

Model 2

- A logistic regression model for one of your binary outcome variables and a

predictor of interest (e.g. predicting whether a tweet received at least 1 like as function of the number of a user's followers)

```
- **Outcome Variable**: Is_Champion
- **Predictor**: Years_Of_Exerience
```

In [14]:
```
est_mod_2 = smf.logit("Is_Champion ~ Years_of_Experience", data=df_f1_final_data).fit()
print(est_mod_2.summary())
```

```
Optimization terminated successfully.
         Current function value: 0.198198
         Iterations 8
                           Logit Regression Results
==============================================================================
Dep. Variable:            Is_Champion   No. Observations:                   24
Model:                          Logit   Df Residuals:                       22
Method:                           MLE   Df Model:                            1
Date:                Sat, 04 Jan 2025   Pseudo R-squ.:                  0.4740
Time:                        19:20:12   Log-Likelihood:                -4.7568
converged:                       True   LL-Null:                       -9.0425
Covariance Type:            nonrobust   LLR p-value:                  0.003415
==========================================================================================
                             coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------------------
Intercept                 -6.2931      2.838     -2.217      0.027     -11.856      -0.730
Years_of_Experience        0.4032      0.216      1.863      0.062      -0.021       0.827
==========================================================================================
```

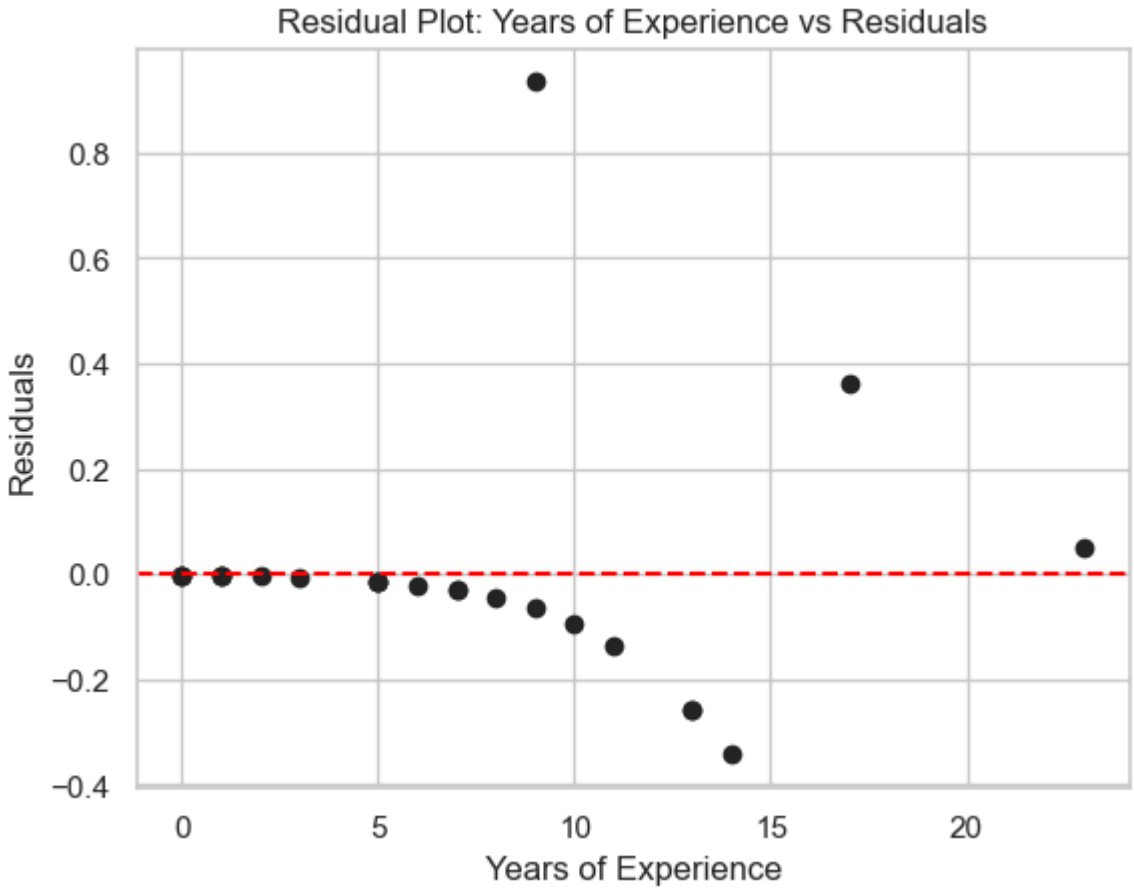
- **Interpretation of Results**:
  - **Intercept = -6.29**: This means that when `Years_of_Experience = 0`, the log-odds of being a champion are -6.29. Converting to odds, $e^{-6.29} \approx 0.00185$, indicating that the odds of being a champion with no experience are extremely low (less than 0.2%). The p-value of 0.027 shows that this result is statistically significant at the $\alpha = 0.05$ level.
  - **Years_of_Experience = 0.4032**: For each additional year of experience, the odds of being a champion increase by approximately $e^{0.4032} - 1 \approx 0.496$ or 49.6%. The p-value is 0.062, which is greater than $\alpha = 0.05$. This means there is insufficient evidence to conclude that years of experience significantly affect the odds of being a champion.

- **Squared Term And Log Transformation**

As in model 2 it will be checked if any non-linear relationships exist between the predictor variable and the outcome variable.

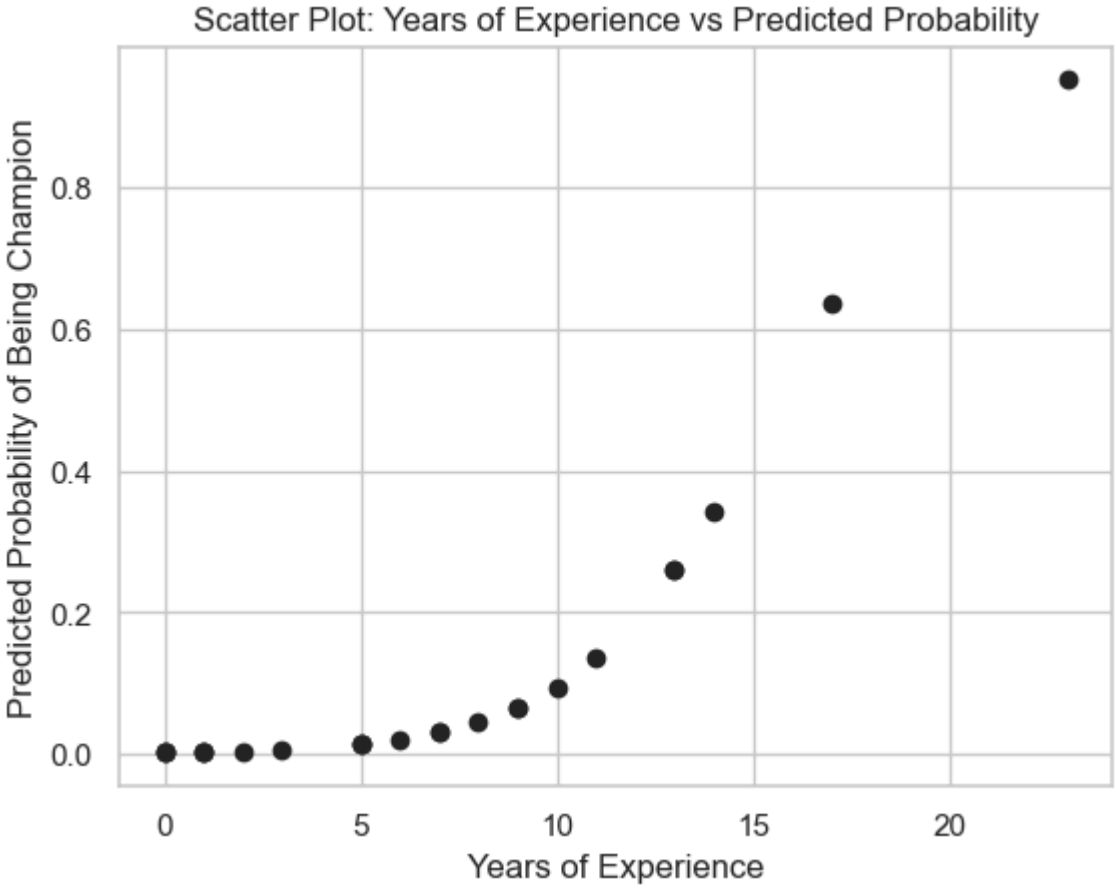
In [15]:
```
#Residual Plot for model 2 (note: logistic regression library does not have resid function, s


# Manually compute the residuals for logistic regression
# Logistic regression residuals are calculated as the difference between the observed outcome
predicted_values = est_mod_2.predict()  # Predicted probabilities
observed_values = df_f1_final_data['Is_Champion']  # Actual values (0 or 1)

# Computing the residuals (observed - predicted)
residuals = observed_values - predicted_values

# Create a scatter plot of residuals vs 'Years_of_Experience'
# This helps us visually inspect if the residuals follow a random pattern or show any non-lin
plt.scatter(df_f1_final_data['Years_of_Experience'], residuals)

# Adding a horizontal line at 0 to represent the baseline where residuals should ideally be c
plt.axhline(0, color='red', linestyle='--')

# Labeling the axes for clarity
plt.xlabel('Years of Experience')
plt.ylabel('Residuals')

# Adding a title to the plot to indicate what it represents
plt.title('Residual Plot: Years of Experience vs Residuals')
```

```
# Displaying the plot to inspect the pattern of residuals
plt.show()
```



The residual plot reveals a non-linear relationship between the predictor variable ('Years of Experience') and the outcome variable ('Is_Champion'). To further investigate whether the data points exhibit any systematic pattern or are skewed (and according to residual plot there might be some patterns), a scatter plot will be generated. This plot will help visualize the relationship between 'Years of Experience' and 'Is_Champion' more clearly and provide insights into the distribution and potential irregularities in the data.

In [16]:
```
# Scatter plot of 'Years_of_Experience' vs predicted probabilities
plt.scatter(df_f1_final_data['Years_of_Experience'], predicted_values)

# Labeling the axes for clarity
plt.xlabel('Years of Experience')
plt.ylabel('Predicted Probability of Being Champion')

# Adding a title to the plot
plt.title('Scatter Plot: Years of Experience vs Predicted Probability')

# Display the plot
plt.show()
```



On the Scatter Plot there is a pattern that resembles an exponential function. This suggests (same as Residual Plot)that the relationship between the predictor (Years_of_Experience) and the outcome variable (Is_Champion) may not be linear, but rather follows a multiplicative or exponential trend. In such case, this pattern may indicate that a log transformation could help improve the model fit by linearizing the relationship.

Model 3

- Conduct a linear or logistic model as above, but include a control and explain

what happens to the coefficient on your predictor of interest when the control is included (e.g. predicting the number of likes a tweet receives as a function of the number of a user's followers, controlling for whether the user is "verified" or not)

```
- **Outcome Variable:** Points
- **Predictors:** Years_Of_Experience, Pole_Positions_Count
```

In [17]:
```
est_mod_3 = ols('Points ~ Years_of_Experience + Pole_Positions_Count',data=df_f1_final_data).
print(np.sqrt(est_mod_3.scale))
est_mod_3.summary().tables[1]
```

```
90.99636004839965
```

Out[17]:

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	38.5973	30.467	1.267	0.219	-24.762	101.957
Years_of_Experience	3.4537	3.180	1.086	0.290	-3.159	10.067
Pole_Positions_Count	47.7307	7.985	5.977	0.000	31.124	64.337

- **Interpretation of Results:**

$$Y_i = 38.60 + 3.45X_{i_1} + 47.73X_{i_2} + \epsilon_i$$

- **Intercept = 38.60**: This represents the estimated number of points a Formula 1 driver is expected to score in the 2024 season when they have 0 years of experience and 0 pole positions. The p-value for the intercept is 0.219, which is greater than the typical significance threshold of 0.05. This suggests that the intercept is not statistically significant, and the baseline value of 38.60 points for a driver with no experience or pole positions may be unreliable. It is likely due to random chance rather than reflecting a true relationship.

- **Years_Of_Experience = 3.45**: For each additional year of Formula 1 experience, the model predicts an increase of approximately 3.45 points in the total score while keeping the Pole_Positions_Count constant. However, the p-value for this coefficient is 0.290, which is much higher than 0.05. This means there is insufficient evidence to consider the relationship between years of experience and points as statistically significant. The observed effect of 3.45 points could likely be due to random variation, and there is no strong evidence to suggest that years of experience significantly impact the number of points scored.

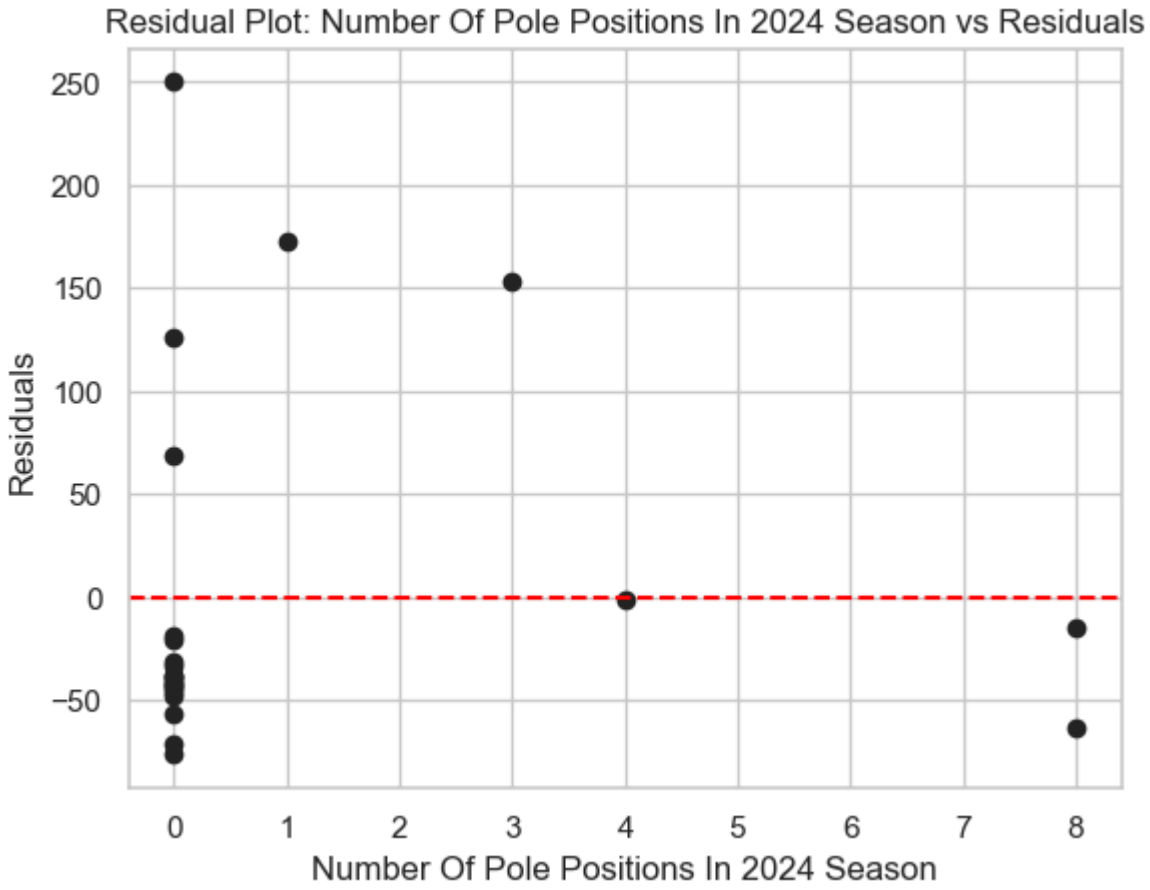
- **Pole_Position_Count = 47.73**: For each additional pole position a driver earns, the model predicts an increase of 47.73 points in their total score while keeping the Years_Of_Experience constant. The p-value for this coefficient is 0.000, which is less than 0.05. This indicates that the relationship between the number of pole positions and the points scored is statistically significant.

- **RMSE (Root Mean Squared1.00r = 90.996)**: The RMSE value of 90.996 indicates that, on average, the model's predictions deviate by about 90.996 points from the actual values. While the model explains some of the variation in points, the RMSE suggests that the model's predictions are still quite far off from the actual values. This indicates that the model could benefit from additional predictors or a different modeling approach to improve its predictive accuracy.

- **Squared Term And Log Transformation**

As in model 1 and 2 it will be checked if any non-linear relationships exist between the predictor variables and the outcome variable. In this model there are 2 predictors, but the first one - Years_Of_Experience was investigated in model 1 and results are available in section `Model 1`.

In [18]:
```
#Analysis for a variable Pole_Position_Count

# Performing a Residual Plot to check if residuals are randomly scattered around the horizonta
# If the residuals are randomly scattered around the horizontal line at 0, it suggests that th
# between the predictor and outcome variable is linear.
residuals = est_mod_3.resid  # Extracting the residuals from the fitted regression model

# Creating a scatter plot of residuals vs 'Years_of_Experience'
# This helps us visually inspect if the residuals follow a random pattern or exhibit any spec
plt.scatter(df_f1_final_data['Pole_Positions_Count'], residuals)

# Adding a horizontal line at 0 to represent the baseline where residuals should be centered
# The line helps us see if residuals are evenly distributed around 0 (indicating a linear rela
plt.axhline(0, color='red', linestyle='--')

# Labeling the x-axis as 'Years of Experience' and the y-axis as 'Residuals' for clarity
plt.xlabel('Number Of Pole Positions In 2024 Season')
plt.ylabel('Residuals')

# Adding a title to the plot to indicate what it represents
plt.title('Residual Plot: Number Of Pole Positions In 2024 Season vs Residuals')

# Displaying the plot
plt.show()
```



The data points in the residual plot do not appear to be randomly scattered around the horizontal line. There is a notable cluster of drivers with zero pole positions in the season, which creates a vertical line. This pattern suggests a potential non-linear relationship between the outcome variable and the given predictor. Similarly as in Model 1 data does not follow symetric or U-shape pattern. Because of that I would not apply squared term or log transformation to this model. However, I would consider applying other non-linear tranformations and also including other predictors in this model to see if there are any interaction effects.

Model 4

- Conduct a linear or logistic regression model as above, but include an interaction

between your predictor of interest and another variable (e.g. predicting the number of likes a tweet receives as a function of the number of a user's followers, whether the user is verified or not, and the interaction between the number of followers and being verified)

```
- **Outcome Variable:** Points
- **Predictors:** Years_Of_Experience, Pole_Positions_Count
```

In [19]:
```
est_mod_4 = ols('Points ~ Years_of_Experience * Pole_Positions_Count',data=df_f1_final_data).
print(np.sqrt(est_mod_4.scale))
est_mod_4.summary().tables[1]
```

```
93.02184707116973
```

Out[19]:

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	39.6688	31.338	1.266	0.220	-25.700	105.038
Years_of_Experience	3.3284	3.276	1.016	0.322	-3.505	10.162
Pole_Positions_Count	39.5711	27.645	1.431	0.168	-18.096	97.239
Years_of_Experience:Pole_Positions_Count	1.2038	3.897	0.309	0.761	-6.925	9.332

- **Interpretation of Results:**

$$Y_i = 39.67 + 3.33X_{i_1} + 39.57X_{i_2} + 1.20X_{i_1}X_{i_2} + \epsilon_i$$

- **Intercept = 39.67**: This represents the expected number of points a Formula 1 driver is predicted to score in the 2024 season when both Years_of_Experience and Pole_Positions_Count are equal to 0. The p-value for the intercept is 0.220, which is greater than the typical significance threshold of 0.05. This suggests that the intercept is not statistically significant, meaning that the baseline value of 39.67 points may be unreliable and could be due to random variation.

- **Years_Of_Experience = 3.33**: For each additional year of Formula 1 experience, the model predicts an increase of approximately 3.33 points in the total score, assuming that the number of pole positions remains constant. However, the p-value for this coefficient is 0.322, which is much higher than 0.05. This indicates insufficient evidence to consider the relationship between years of experience and points as statistically significant.

- **Pole_Position_Count = 47.73**: For each additional pole position a driver earns, the model predicts an increase of approximately 39.57 points in their total score, assuming that the number of years of experience remains constant. The p-value for this coefficient is 0.168, which is also greater than 0.05, suggesting that there is no strong evidence to consider this relationship statistically significant.

- **Interaction Term (Years_of_Experience:Pole_Positions_Count) = 1.20**: The interaction term represents how the effect of Years_of_Experience on the number of points changes depending on the number of pole positions a driver has. Specifically, for each additional pole position, the effect of Years_of_Experience on the total points increases by 1.20 points. However, the p-value for the interaction term is 0.761, which is much greater than 0.05. This indicates that the interaction effect is not statistically significant, meaning that there is no strong evidence to suggest that the combined effect of years of experience and pole positions significantly impacts the number of points scored.

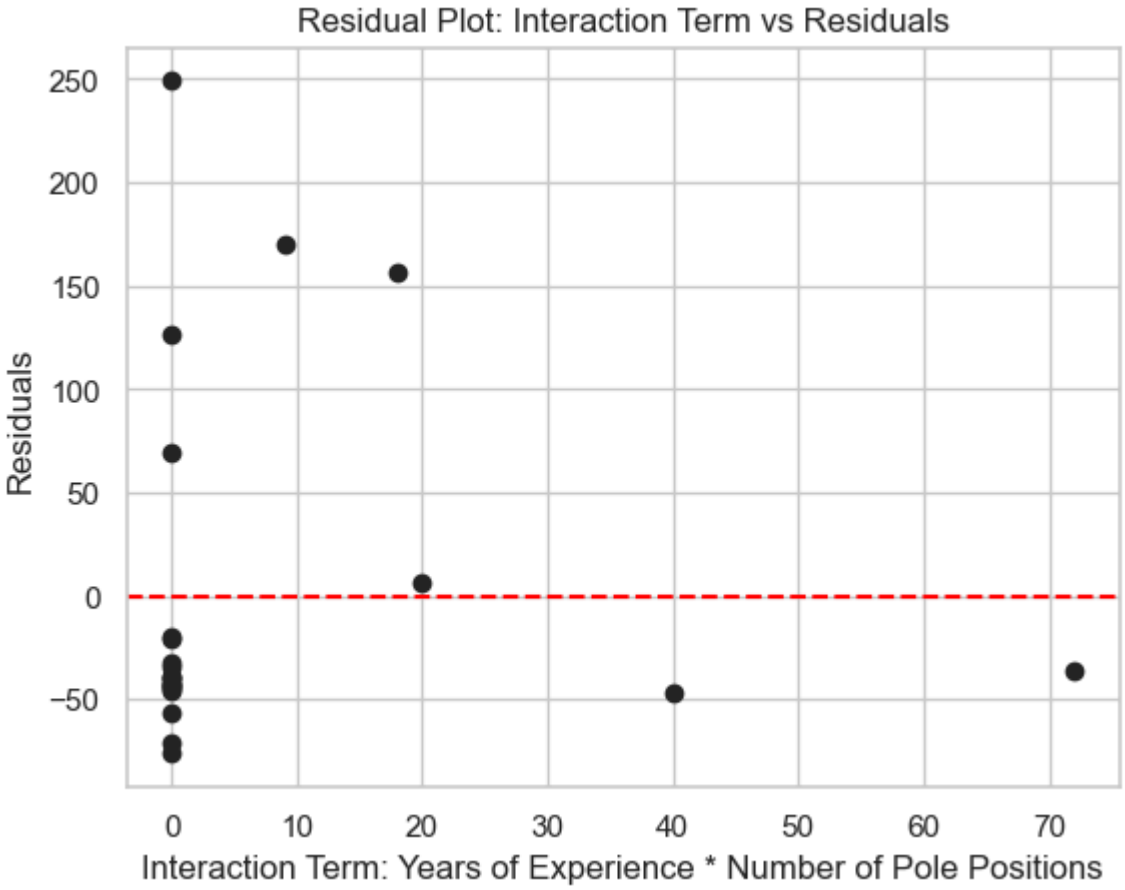
- **RMSE (Root Mean Squared1.00r = 93.02)**: The RMSE value of 93.02 indicates that, on average, the model's predictions deviate by about 93.02 points from the actual values.

- **Squared Term And Log Transformation**

In `Model 1` and `Model 3` sections linearity, squared term, and log transformation between outcome variable (Points) and both predictors (Years_of_Experience, Pole_Positions_Count) were discussed. In this section the main focus would be on the interaction term Years_of_Experience * Pole_Positions_Count.

In [20]:
```
# Analysis for the interaction term: Years_of_Experience * Pole_Positions_Count

# Performing a Residual Plot to check if residuals are randomly scattered around the horizonta
# If the residuals are randomly scattered around the horizontal line at 0, it suggests that th
# between the predictor and outcome variable is linear.

# Extracting the residuals from the fitted regression model
residuals = est_mod_4.resid

# Creating a new variable for the interaction term (Years_of_Experience * Pole_Positions_Count
# This variable will be used on the x-axis of the residual plot.
interaction_term = df_f1_final_data['Years_of_Experience'] * df_f1_final_data['Pole_Positions_

# Creating a scatter plot of residuals vs the interaction term
# This helps us visually inspect if the residuals follow a random pattern or exhibit any spec
plt.scatter(interaction_term, residuals)
```

```
# Adding a horizontal line at 0 to represent the baseline where residuals should be centered
# The line helps us see if residuals are evenly distributed around 0 (indicating a linear rela
plt.axhline(0, color='red', linestyle='--')

# Labeling the x-axis as the interaction term and the y-axis as 'Residuals' for clarity
plt.xlabel('Interaction Term: Years of Experience * Number of Pole Positions')
plt.ylabel('Residuals')

# Adding a title to the plot to indicate what it represents
plt.title('Residual Plot: Interaction Term vs Residuals')

# Displaying the plot
plt.show()
```



In this case, the situation is similar to the one described in `Model 3`. The data points in the residual plot do not appear to be randomly scattered around the horizontal line, suggesting potential issues with the model fit. However, the interaction term is not statistically significant, and adding it to the model does not improve its explanatory power. Therefore, any log transformations or the inclusion of squared terms should not be applied in this case, as they would unnecessarily complicate the model without addressing the lack of statistical significance.

Model 5

- Conduct a linear or logistic regression model for any outcome of interest with

a non- binary categorical variable (a variable with more than 2 categories) as the predictor

```
    - **Outcome Variable:** Finished_Races_Percentage
    - **Predictor:** Constructor
```

In [21]:
```
est_mod_5 = ols('Finished_Races_Percentage ~ C(Constructor)',data=df_f1_final_data).fit()
print(np.sqrt(est_mod_5.scale))
est_mod_5.summary().tables[1]
```

```
14.325258619787968
```

Out[21]:

	coef	std err	t	P>|t|	[0.025	0.975]
Intercept	25.4200	8.271	3.074	0.008	7.681	43.159
C(Constructor)[T.Aston Martin]	28.7500	13.077	2.198	0.045	0.702	56.798
C(Constructor)[T.Ferrari]	67.4567	11.697	5.767	0.000	42.370	92.543
C(Constructor)[T.Haas F1 Team]	28.9400	13.077	2.213	0.044	0.892	56.988
C(Constructor)[T.McLaren]	72.4950	13.077	5.544	0.000	44.447	100.543
C(Constructor)[T.Mercedes]	64.1650	13.077	4.907	0.000	36.117	92.213
C(Constructor)[T.RB F1 Team]	20.4133	11.697	1.745	0.103	-4.673	45.500
C(Constructor)[T.Red Bull]	55.8300	13.077	4.269	0.001	27.782	83.878
C(Constructor)[T.Sauber]	-2.5050	13.077	-0.192	0.851	-30.553	25.543
C(Constructor)[T.Williams]	6.2600	11.697	0.535	0.601	-18.827	31.347

- **Interpretation of Results:**

$$Y_i = 25.42 + 28.75X_{i_2} + 67.46X_{i_3} + 28.94X_{i_4} + 72.50X_{i_5} + 64.17X_{i_6} + 20.41X_{i_7} + 55.83X_{i_8} - 2.51$$

- **Intercept = 25.42**: The intercept represents the expected percentage of races finished for a driver (observation i) from the reference constructor (Alpine). The model predicts that, on average, a driver from Alpine is expected to finish 25.42% of races. This coefficient is statistically significant with a p-value of 0.008, indicating strong evidence that the average finished race percentage for Alpine drivers is different from zero.

- **C(Constructor)[T.Aston Martin] = 28.75**: A driver i from Aston Martin is predicted to finish 28.75% more races compared to driver j from Alpine. The p-value for this coefficient is 0.045, which is below the significance threshold of 0.05, suggesting that this difference is statistically significant.

- **C(Constructor)[T.Ferrari] = 67.46**: A driver i from Ferrari is predicted to finish 67.46% more races compared to a driver j from Alpine. The p-value for this coefficient is less than 0.001, indicating that this difference is highly statistically significant.

- **C(Constructor)[T.Haas F1 Team] = 28.94**: A driver i from Haas F1 Team is predicted to finish 28.94% more races compared to a driver j from Alpine. The p-value for this coefficient is 0.044, indicating that this difference is statistically significant.

- **C(Constructor)[T.McLaren] = 72.50**: A driver i from McLaren is predicted to finish 72.50% more races compared to a driver j from Alpine. The p-value for this coefficient is less than 0.001, indicating that this difference is highly statistically significant.

- **C(Constructor)[T.Mercedes] = 64.17**: A driver i from Mercedes is predicted to finish 64.17% more races compared to a driver j from Alpine. The p-value for this coefficient is less than 0.001, indicating strong statistical significance.

- **C(Constructor)[T.RB F1 Team] = 20.41**: A driver i from RB F1 Team is predicted to finish 20.41% more races compared to a driver j from Alpine. However, the p-value for this coefficient is 0.103, which is greater than 0.05. This suggests that there is no strong evidence to conclude that this difference is statistically significant.

- **C(Constructor)[T.Red Bull] = 55.83**: A driver from Red Bull is predicted to finish 55.83% more races compared to a driver from Alpine. The p-value for this coefficient is 0.001, indicating strong

statistical significance.

- **C(Constructor)[T.Sauber] = -2.51**: A driver i from Sauber is predicted to finish 2.51% fewer races compared to a driver j from Alpine. The p-value for this coefficient is 0.851, which is much greater than 0.05, indicating that the difference is not statistically significant.

- **C(Constructor)[T.Williams] = 6.26**: A driver i from Williams is predicted to finish 6.26% more races compared to a driver j from Alpine. The p-value for this coefficient is 0.601, indicating that this difference is not statistically significant.

- **Root Mean Squared Error (RMSE) = 14.33**: The RMSE value of 14.33 indicates that, on average, the model's predictions deviate by about 14.33 percentage points from the actual values of the finished races percentage.

- **Squared Term And Log Transformation**

In this case, there is no need to apply a log transformation or add a squared term because the predictor variable Constructor is categorical. That means it does not have a continuous or ordered scale. Categorical variables are usually handled through dummy variables (as in this case), where each category is represented as a binary variable.